

OB - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/", RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4363 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4212 values"
## [1] "151 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	541	35	131	NA	NA	707
2	67	NA	420	98	107	692
3	305	NA	163	202	15	685
4	297	NA	233	67	36	633
5	232	NA	203	70	33	538
6	201	1	117	65	22	406
7	161	NA	99	26	18	304
8	91	NA	48	25	4	168
9	70	NA	2	NA	7	79

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7652051	0.0495050	0.1852900	NA	NA	707
2	0.0968208	NA	0.6069364	0.1416185	0.1546243	692
3	0.4452555	NA	0.2379562	0.2948905	0.0218978	685
4	0.4691943	NA	0.3680885	0.1058452	0.0568720	633
5	0.4312268	NA	0.3773234	0.1301115	0.0613383	538
6	0.4950739	0.0024631	0.2881773	0.1600985	0.0541872	406
7	0.5296053	NA	0.3256579	0.0855263	0.0592105	304
8	0.5416667	NA	0.2857143	0.1488095	0.0238095	168
9	0.8860759	NA	0.0253165	NA	0.0886076	79

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=sy
```

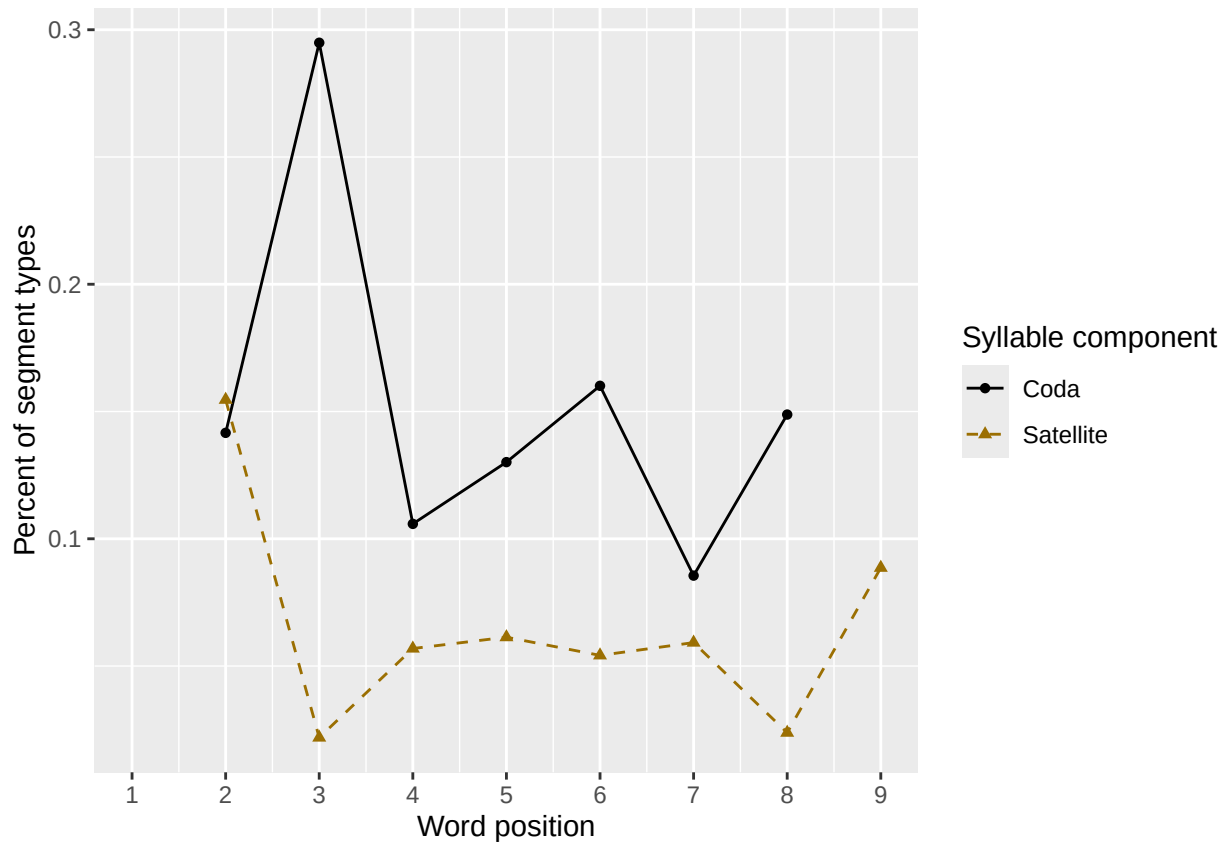
Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4 0.984 0.967 0.967 NA     NA     NA     NA     NA     NA
## 2     5 0.978 0.968 0.925 0.882 NA     NA     NA     NA     NA
## 3     6 0.992 0.941 0.915 0.890 0.771 NA     NA     NA     NA
## 4     7 0.982 0.964 0.945 0.857 0.879 0.745 NA     NA     NA
## 5     8 0.966 0.914 0.863 0.820 0.711 0.711 0.591 NA     NA
## 6     9 0.947 0.924 0.886 0.844 0.791 0.717 0.655 0.593 NA
## 7    10 0.939 0.923 0.948 0.753 0.738 0.684 0.638 0.610 0.544

```

```
# len/pos table
```

```
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                                "_N_by_len_position.csv"))

```

```
pos_len_N_table
```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1     4    61    60    60    NA    NA    NA    NA    NA    NA
## 2     5    93    93    93    93    NA    NA    NA    NA    NA
## 3     6   118   118   118   118   118    NA    NA    NA    NA
## 4     7   112   111   110   112   107   110    NA    NA    NA
## 5     8   147   140   139   139   142   128   137    NA    NA
## 6     9    94    92    88    90    91    92    87    91    NA
## 7    10    82    78    77    81    80    76    80    77    79

```

```

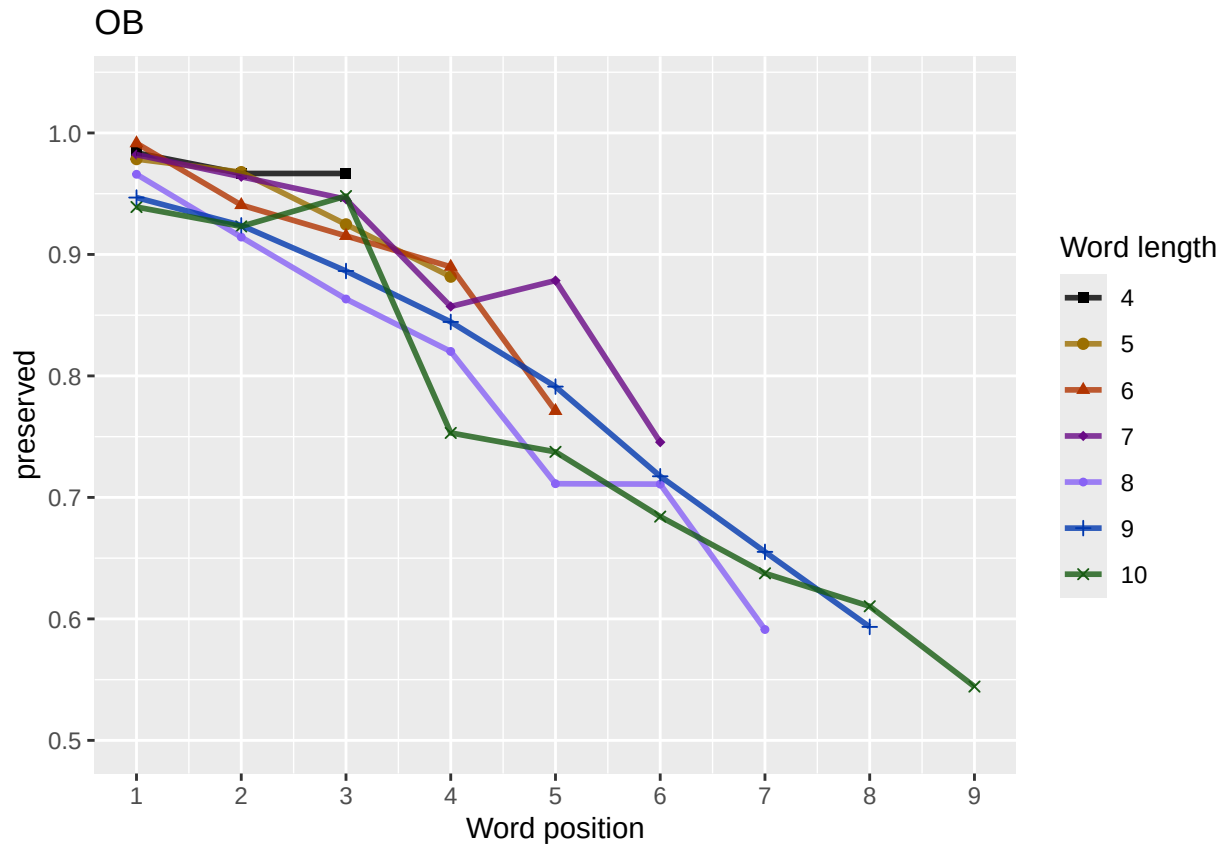
obs_linetypes <- c("solid", "solid", "solid", "solid",
                  "solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved, max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)

```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 8
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      6.1869902      -0.2767346      0.0247210      -0.9810918      0.0003683      0.0003683
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4206 Residual
## Null Deviance:      3701
## Residual Deviance: 3227  AIC: 3239
## log likelihood: -1613.431
## Nagelkerke R2:  0.1820015
## % pres/err predicted correctly: -1002.879
## % of predictable range [ (model-null)/(1-null) ]:  0.112345
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos
##      5.15904      -0.12328      0.04249      -0.79707
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4208 Residual
## Null Deviance:      3701
## Residual Deviance: 3231  AIC: 3239
## log likelihood: -1615.459
## Nagelkerke R2:  0.1805292
## % pres/err predicted correctly: -1003.427
## % of predictable range [ (model-null)/(1-null) ]:  0.1118604
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      pos  stimlen:pos
##      6.45469      -0.36950      -0.96828      0.06672
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4208 Residual
## Null Deviance:      3701
## Residual Deviance: 3232  AIC: 3240
## log likelihood: -1616.218
## Nagelkerke R2:  0.1799777
## % pres/err predicted correctly: -1002.304
## % of predictable range [ (model-null)/(1-null) ]:  0.112854
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      I(pos^2)      pos

```

```

##      4.25999      0.03635      -0.78085
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4209 Residual
## Null Deviance:      3701
## Residual Deviance: 3245 AIC: 3251
## log likelihood: -1622.255
## Nagelkerke R2: 0.1755836
## % pres/err predicted correctly: -1006.09
## % of predictable range [ (model-null)/(1-null) ]: 0.1095059
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos
##      4.12963      -0.09682      -0.38599
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4209 Residual
## Null Deviance:      3701
## Residual Deviance: 3250 AIC: 3256
## log likelihood: -1625.1
## Nagelkerke R2: 0.1735094
## % pres/err predicted correctly: -1002.831
## % of predictable range [ (model-null)/(1-null) ]: 0.1123878
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.5203      -0.4207
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3259 AIC: 3263
## log likelihood: -1629.497
## Nagelkerke R2: 0.1702967
## % pres/err predicted correctly: -1005.036
## % of predictable range [ (model-null)/(1-null) ]: 0.1104375
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.1665      -0.3159
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4210 Residual

```



```
## Null Deviance:      3701
## Residual Deviance: 3559 AIC: 3563
## log likelihood: -1779.279
## Nagelkerke R2: 0.05676984
## % pres/err predicted correctly: -1094.075
## % of predictable range [ (model-null)/(1-null) ]: 0.0317071
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.66
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4211 Residual
## Null Deviance:      3701
## Residual Deviance: 3701 AIC: 3703
## log likelihood: -1850.363
## Nagelkerke R2: -3.797957e-16
## % pres/err predicted correctly: -1129.934
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	3238.862	0.0000000	0.0000000	0.0411317	0.1182006	5186990	-	-	0.0372620	0.0247200	0.0003683
							0.2767346	0.9810918			
preserved ~ stimlen + I(pos^2) + pos	3238.917	0.053724	0.9726936	0.4000855	0.1805292	5159037	-	-	NA	0.0424892	NA
							0.1232795	0.7970707			
preserved ~ stimlen * pos	3240.435	0.5732070	0.4553887	0.1873092	0.1799767	454691	-	-	0.0667215	NA	NA
							0.3695031	0.9682806			
preserved ~ I(pos^2) + pos	3250.511	1.6490373	0.0295420	0.0121511	0.1755846	259987	NA	-	NA	0.0363470	NA
								0.7808527			

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~	3256.1997	337270	0.000170	0.000070	0.717350	4129630	-	-	NA	NA
stimlen + pos							0.0968196	0.3859865		
preserved ~ pos	3262.9924	1.132510	0.000005	0.000002	0.417029	37520265	NA	-	NA	NA
								0.4207398		
preserved ~	3562.5537	23.695583	0.000000	0.000000	0.005676	48166546	-	NA	NA	NA
stimlen							0.3158916			
preserved ~ 1	3702.7263	8.864572	0.000000	0.000000	0.000000	00659854	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2)
```

```
## 6.1869902 -0.2767346 0.0247210 -0.9810918 0.0003683 0.0003683
```

```
##
```

```
## Degrees of Freedom: 4211 Total (i.e. Null); 4206 Residual
```

```
## Null Deviance: 3701
```

```
## Residual Deviance: 3227 AIC: 3239
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups: stimlen [7]
```

```
## stimlen `1` `2` `3` `4` `5` `6` `7` `8` `9`
```

```
## <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1 4 0.986 0.971 0.944 NA NA NA NA NA NA
```

```
## 2 5 0.983 0.965 0.935 0.886 NA NA NA NA NA
```

```
## 3 6 0.978 0.958 0.924 0.873 0.804 NA NA NA NA
```

```
## 4 7 0.972 0.949 0.912 0.859 0.791 0.713 NA NA NA
```

```
## 5 8 0.965 0.938 0.898 0.843 0.777 0.705 0.634 NA NA
```

```
## 6 9 0.956 0.925 0.882 0.827 0.763 0.697 0.634 0.581 NA
```

```
## 7 10 0.945 0.910 0.864 0.808 0.748 0.688 0.635 0.591 0.561
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```

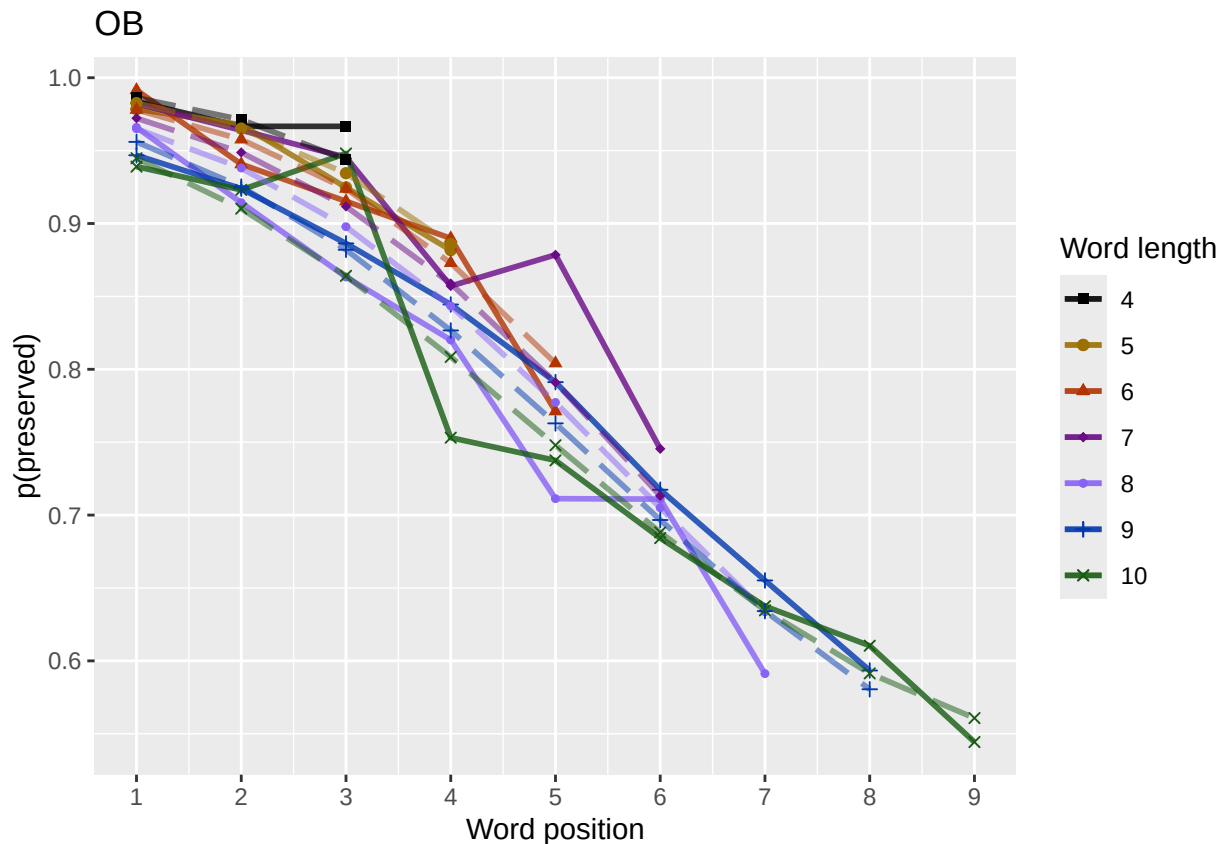
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen, color=stimlen))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient_id))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num <- 0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1     138    711

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 138 / 711 = 19.41 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    5.60985    -0.20661     0.05611    -0.67620
##
## Degrees of Freedom: 3857 Total (i.e. Null);  3854 Residual
## Null Deviance:      2201
## Residual Deviance: 2096  AIC: 2104
## log likelihood:  -1048.085
## Nagelkerke R2:   0.06177838
## % pres/err predicted correctly:  -569.0824
## % of predictable range [ (model-null)/(1-null) ]:  0.02591102
## *****

```

```

## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          I(pos^2)          pos  stimlen:I(pos^2)          stimlen:I(pos^2)
##      5.640788      -0.232701      -0.021287      -0.356976          0.007419          -0.007419
##
## Degrees of Freedom: 3857 Total (i.e. Null); 3852 Residual
## Null Deviance:      2201
## Residual Deviance: 2094 AIC: 2106
## log likelihood: -1046.938
## Nagelkerke R2: 0.06310883
## % pres/err predicted correctly: -568.8378
## % of predictable range [ (model-null)/(1-null) ]: 0.02632895
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          pos  stimlen:pos
##      6.67938      -0.43451      -0.79172          0.07113
##
## Degrees of Freedom: 3857 Total (i.e. Null); 3854 Residual
## Null Deviance:      2201
## Residual Deviance: 2103 AIC: 2111
## log likelihood: -1051.689
## Nagelkerke R2: 0.05759257
## % pres/err predicted correctly: -570.4257
## % of predictable range [ (model-null)/(1-null) ]: 0.02361572
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          pos
##      4.4654      -0.1758      -0.1705
##
## Degrees of Freedom: 3857 Total (i.e. Null); 3855 Residual
## Null Deviance:      2201
## Residual Deviance: 2115 AIC: 2121
## log likelihood: -1057.584
## Nagelkerke R2: 0.05072964
## % pres/err predicted correctly: -571.5499
## % of predictable range [ (model-null)/(1-null) ]: 0.02169486
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      4.05693      0.04519      -0.63955
##
## Degrees of Freedom: 3857 Total (i.e. Null);  3855 Residual
## Null Deviance:      2201
## Residual Deviance: 2122 AIC: 2128
## log likelihood:  -1060.777
## Nagelkerke R2:  0.04700387
## % pres/err predicted correctly:  -572.6493
## % of predictable range [ (model-null)/(1-null) ]:  0.01981633
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.3017      -0.2251
##
## Degrees of Freedom: 3857 Total (i.e. Null);  3856 Residual
## Null Deviance:      2201
## Residual Deviance: 2134 AIC: 2138
## log likelihood:  -1067.068
## Nagelkerke R2:  0.03964437
## % pres/err predicted correctly:  -574.0658
## % of predictable range [ (model-null)/(1-null) ]:  0.01739608
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.4309      -0.2575
##
## Degrees of Freedom: 3857 Total (i.e. Null);  3856 Residual
## Null Deviance:      2201
## Residual Deviance: 2150 AIC: 2154
## log likelihood:  -1075.14
## Nagelkerke R2:  0.03016592
## % pres/err predicted correctly:  -577.0098
## % of predictable range [ (model-null)/(1-null) ]:  0.01236565
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```

## (Intercept)
##      2.406
##
## Degrees of Freedom: 3857 Total (i.e. Null);  3857 Residual
## Null Deviance:      2201
## Residual Deviance: 2201  AIC: 2203
## log likelihood:  -1100.608
## Nagelkerke R2:   5.106934e-16
## % pres/err predicted correctly:  -584.2468
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	2104.170	0.000000	1.000000	0.0068796	0.0617784	609847	-	-	NA	0.0561066	NA
							0.2066116	6762015			
preserved ~ stimlen * (I(pos^2) + pos)	2105.876	1.705993	0.4261362	0.2931679	0.0631088	640788	-	-	-	-	0.0074193
							0.2327008	3569735	0.0244486	0.0212874	
preserved ~ stimlen * pos	2111.377	7.208442	0.0272086	0.0187187	0.0575926	679377	-	-	0.0711297	NA	NA
							0.4345105	7917210			
preserved ~ stimlen + pos	2121.168	6.998109	0.0000203	0.0001401	0.0507296	6465375	-	-	NA	NA	NA
							0.1758412	1704716			
preserved ~ I(pos^2) + pos	2127.552	3.383457	0.0000084	0.0000058	0.0470039	56934	NA	-	NA	0.0451934	NA
								0.6395484			
preserved ~ pos	2138.136	3.965469	0.0000000	0.0000000	0.0396434	301706	NA	-	NA	NA	NA
								0.2251252			
preserved ~ stimlen	2154.285	10.109856	0.0000000	0.0000000	0.0301639	430926	-	NA	NA	NA	NA
							0.2574810				
preserved ~ 1	2203.219	99.045593	0.0000000	0.0000000	0.0000000	20406408	NA	NA	NA	NA	NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.985 0.975 0.963 NA      NA      NA      NA      NA      NA
## 2      5 0.981 0.969 0.955 0.941 NA      NA      NA      NA      NA
## 3      6 0.977 0.962 0.945 0.928 0.916 NA      NA      NA      NA
## 4      7 0.972 0.954 0.933 0.913 0.899 0.893 NA      NA      NA
## 5      8 0.966 0.944 0.919 0.896 0.879 0.872 0.878 NA      NA
## 6      9 0.958 0.932 0.903 0.875 0.855 0.847 0.854 0.873 NA
## 7     10 0.949 0.918 0.883 0.850 0.827 0.819 0.826 0.849 0.881
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
```

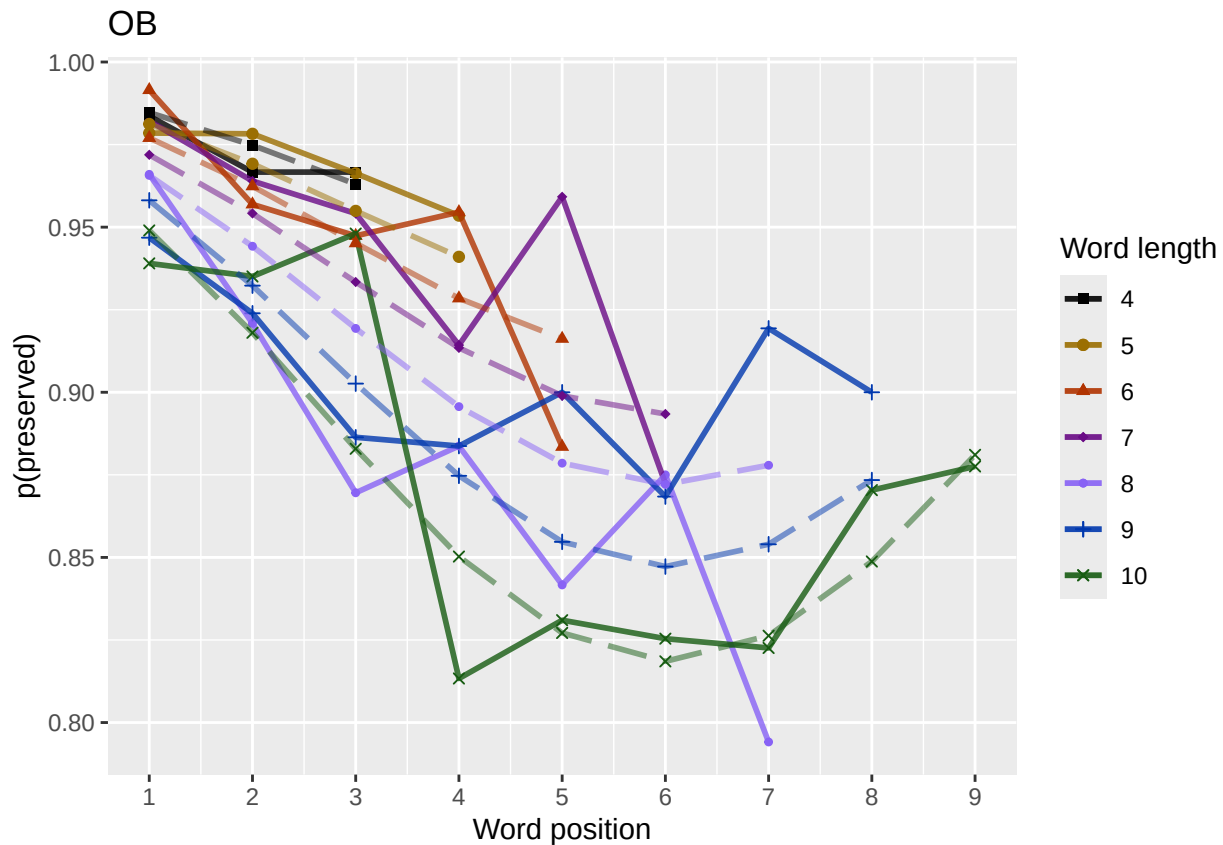
```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted
```

```
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.50 - 1.04"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.009639253
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.04874905
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stimlen:log
##      4.91590      -0.09622      0.33139      0.04095      -0.78232      -0
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4206 Residual
## Null Deviance:      3701
## Residual Deviance: 3214  AIC: 3226
## log likelihood: -1606.933
## Nagelkerke R2:  0.18671
## % pres/err predicted correctly: -998.6734
## % of predictable range [ (model-null)/(1-null) ]:  0.1160639
## *****
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      log_freq
##      4.95458      -0.09693      0.04164      -0.78974      0.08927
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4207 Residual
## Null Deviance:      3701
## Residual Deviance: 3217  AIC: 3227
## log likelihood: -1608.637
## Nagelkerke R2:  0.1854768
## % pres/err predicted correctly: -998.9756
## % of predictable range [ (model-null)/(1-null) ]:  0.1157967
## *****
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stim
##      4.9360824      -0.1002794      0.3619269      0.0418751      -0.7852509
##      log_freq:pos
##      -0.0006077
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4204 Residual
## Null Deviance:      3701
## Residual Deviance: 3213  AIC: 3229
## log likelihood: -1606.465
## Nagelkerke R2:  0.1870485
## % pres/err predicted correctly: -998.0494
## % of predictable range [ (model-null)/(1-null) ]:  0.1166157
## *****
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```

##      (Intercept)      stimlen      I(pos^2)      pos      log_freq I(pos
##      4.9514621      -0.0969567      0.0414722      -0.7881116      0.0779378
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4205 Residual
## Null Deviance:      3701
## Residual Deviance: 3217 AIC: 3231
## log likelihood: -1608.632
## Nagelkerke R2: 0.1854804
## % pres/err predicted correctly: -998.957
## % of predictable range [ (model-null)/(1-null) ]: 0.1158131
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      I(pos^2)      pos      log_freq I(pos^2):log_freq
##      4.2395250      0.0363986      -0.7727291      0.1130928      -0.0006987
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4206 Residual
## Null Deviance:      3701
## Residual Deviance: 3225 AIC: 3237
## log likelihood: -1612.512
## Nagelkerke R2: 0.1826683
## % pres/err predicted correctly: -1000.255
## % of predictable range [ (model-null)/(1-null) ]: 0.1146657
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      stimlen:I(pos^2)      stiml
##      6.1869902      -0.2767346      0.0247210      -0.9810918      0.0003683      0.0
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4206 Residual
## Null Deviance:      3701
## Residual Deviance: 3227 AIC: 3239
## log likelihood: -1613.431
## Nagelkerke R2: 0.1820015
## % pres/err predicted correctly: -1002.879
## % of predictable range [ (model-null)/(1-null) ]: 0.112345
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos
##      5.15904      -0.12328      0.04249      -0.79707
##

```

```

## Degrees of Freedom: 4211 Total (i.e. Null); 4208 Residual
## Null Deviance: 3701
## Residual Deviance: 3231 AIC: 3239
## log likelihood: -1615.459
## Nagelkerke R2: 0.1805292
## % pres/err predicted correctly: -1003.427
## % of predictable range [ (model-null)/(1-null) ]: 0.1118604
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 6.45469 -0.36950 -0.96828 0.06672
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4208 Residual
## Null Deviance: 3701
## Residual Deviance: 3232 AIC: 3240
## log likelihood: -1616.218
## Nagelkerke R2: 0.1799777
## % pres/err predicted correctly: -1002.304
## % of predictable range [ (model-null)/(1-null) ]: 0.112854
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 3.91615 -0.06964 0.34840 -0.38635 -0.03133
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4207 Residual
## Null Deviance: 3701
## Residual Deviance: 3232 AIC: 3242
## log likelihood: -1615.863
## Nagelkerke R2: 0.180235
## % pres/err predicted correctly: -998.1572
## % of predictable range [ (model-null)/(1-null) ]: 0.1165203
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 3.917866 -0.072481 0.350227 -0.382128 -0.037227 0.
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4206 Residual
## Null Deviance: 3701
## Residual Deviance: 3231 AIC: 3243

```

```

## log likelihood: -1615.599
## Nagelkerke R2: 0.1804272
## % pres/err predicted correctly: -997.6544
## % of predictable range [ (model-null)/(1-null) ]: 0.1169649
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq
##      3.93957      -0.06989      -0.38724      0.09228
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4208 Residual
## Null Deviance:      3701
## Residual Deviance: 3236 AIC: 3244
## log likelihood: -1617.855
## Nagelkerke R2: 0.1787876
## % pres/err predicted correctly: -998.3612
## % of predictable range [ (model-null)/(1-null) ]: 0.1163399
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq pos:log_freq
##      3.935729      -0.068638      -0.388683      0.107270      -0.003014
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4207 Residual
## Null Deviance:      3701
## Residual Deviance: 3236 AIC: 3246
## log likelihood: -1617.819
## Nagelkerke R2: 0.1788132
## % pres/err predicted correctly: -998.5129
## % of predictable range [ (model-null)/(1-null) ]: 0.1162058
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      3.4947      -0.4109      0.1031
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4209 Residual
## Null Deviance:      3701
## Residual Deviance: 3240 AIC: 3246
## log likelihood: -1620.016
## Nagelkerke R2: 0.1772148
## % pres/err predicted correctly: -999.2309

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.115571
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq pos:log_freq
##    3.503392    -0.413057    0.133872    -0.006267
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4208 Residual
## Null Deviance: 3701
## Residual Deviance: 3240 AIC: 3248
## log likelihood: -1619.862
## Nagelkerke R2: 0.1773271
## % pres/err predicted correctly: -999.5076
## % of predictable range [ (model-null)/(1-null) ]: 0.1153262
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##    4.25999    0.03635    -0.78085
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4209 Residual
## Null Deviance: 3701
## Residual Deviance: 3245 AIC: 3251
## log likelihood: -1622.255
## Nagelkerke R2: 0.1755836
## % pres/err predicted correctly: -1006.09
## % of predictable range [ (model-null)/(1-null) ]: 0.1095059
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    4.12963    -0.09682    -0.38599
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4209 Residual
## Null Deviance: 3701
## Residual Deviance: 3250 AIC: 3256
## log likelihood: -1625.1
## Nagelkerke R2: 0.1735094
## % pres/err predicted correctly: -1002.831
## % of predictable range [ (model-null)/(1-null) ]: 0.1123878
## *****
## model index: 16

```

```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.5203      -0.4207
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3259  AIC: 3263
## log likelihood:  -1629.497
## Nagelkerke R2:  0.1702967
## % pres/err predicted correctly:  -1005.036
## % of predictable range [ (model-null)/(1-null) ]:  0.1104375
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq  stimlen:log_freq
##      3.96934      -0.29120      0.35651      -0.03286
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4208 Residual
## Null Deviance:      3701
## Residual Deviance: 3540  AIC: 3548
## log likelihood:  -1770.05
## Nagelkerke R2:  0.06400039
## % pres/err predicted correctly:  -1090.335
## % of predictable range [ (model-null)/(1-null) ]:  0.03501427
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq
##      3.99060      -0.29165      0.08554
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4209 Residual
## Null Deviance:      3701
## Residual Deviance: 3545  AIC: 3551
## log likelihood:  -1772.429
## Nagelkerke R2:  0.06213988
## % pres/err predicted correctly:  -1090.738
## % of predictable range [ (model-null)/(1-null) ]:  0.03465775
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```

##
## Coefficients:
## (Intercept)      stimlen
##      4.1665      -0.3159
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3559  AIC: 3563
## log likelihood:  -1779.279
## Nagelkerke R2:  0.05676984
## % pres/err predicted correctly:  -1094.075
## % of predictable range [ (model-null)/(1-null) ]:  0.0317071
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.66
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4211 Residual
## Null Deviance:      3701
## Residual Deviance: 3701  AIC: 3703
## log likelihood:  -1850.363
## Nagelkerke R2:  -3.797957e-16
## % pres/err predicted correctly:  -1129.934
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)

```

Model	AIC Delta	AIC	AICw	NagR	R ²	Intercept	log_stimlen	log_freq	log_pos	log_freq(I(pos^2))	I(pos^2)	log_freq(I(pos^2))	log_freq(I(pos^2))	len:I(pos^2)
preserved ~ stimlen * log_freq + I(pos^2) + pos	3225.866	0.000000	0.000000	0.983	0.915	898	0.3313942	-	NA	NA	0.0409544	NA	NA	NA
							0.0962235	0.0294743	0.7823171					
preserved ~ stimlen + I(pos^2) + pos + log_freq	3227.274	0.705494	0.876942	0.984	0.954	755	0.0892727	-	NA	NA	0.0416394	NA	NA	NA
							0.0969314	0.7897427						
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	3228.336	0.482160	0.820618	0.985	0.985	082	0.3619269	-	NA	-	0.0418751	0.0013424	NA	NA
							0.1002794	0.0376897	0.852509	0.0006077				
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	3231.569	0.709672	0.987663	0.985	0.954	462	0.0779378	-	0.0052091	0.0414722	NA	NA	NA	NA
							0.0969567	0.7881116		0.0005075				
preserved ~ (I(pos^2) + pos) * log_freq	3237.024	0.505083	0.772011	0.986	0.939	521	0.1130028	-	0.0021940	0.0363986	NA	NA	NA	NA
								0.7727291		0.0006987				
preserved ~ stimlen * (I(pos^2) + pos)	3238.832	0.956201	0.667084	0.986	0.985	185	0.2767346	-	NA	NA	0.0247210	NA	0.0372627	0.03683
								0.9810918						
preserved ~ stimlen + I(pos^2) + pos	3238.917	0.509921	0.666828	0.985	0.952	037	NA	NA	-	NA	NA	0.0424892	NA	NA
							0.1232795	0.7970707						
preserved ~ stimlen * pos	3240.445	0.688270	0.862038	0.984	0.975	494	NA	NA	-	NA	NA	NA	NA	0.0667215
							0.3695031	0.9682806						
preserved ~ stimlen * log_freq + pos	3241.728	0.606370	0.839702	0.983	0.956	146	0.3483984	-	NA	NA	NA	NA	NA	NA
							0.0696430	0.0313323	0.863511					
preserved ~ stimlen * log_freq + pos	3243.178	0.307700	0.720096	0.984	0.972	866	0.3502267	-	NA	0.0093180	NA	NA	NA	NA
							0.0724806	0.0372263	0.821284					
preserved ~ stimlen + pos + log_freq	3243.778	0.490000	0.850074	0.983	0.939	567	0.0922841	-	NA	NA	NA	NA	NA	NA
							0.0698883	0.3872413						
preserved ~ stimlen + pos * log_freq	3245.637	0.243300	0.500028	0.985	0.935	729	0.1072697	-	-	NA	NA	NA	NA	NA
							0.0686380	0.3886329	0.0141					

Model	AIC Delta	AIC	AICw	NagR ²	(Intercept)	log_stimlen	log_freq	log_pos	I(pos^2)	pos	stimlen:log_freq	len:I(pos^2)
preserved ~ pos + log_freq	3246.232	6589800	0.0802	2773.448	0.103061	-	NA	NA	NA	NA	NA	NA
						0.4109174						
preserved ~ pos * log_freq	3247.224	5732300	0.0901	1073.270	0.1338721	-	-	NA	NA	NA	NA	NA
						0.4130570	0.62675					
preserved ~ I(pos^2) + pos	3250.216	4403700	0.0900	1254.239	0.0871	NA	NA	-	NA	NA	0.0363470	NA
								0.7808527				NA
preserved ~ stimlen + pos	3256.393	3289100	0.0800	1073.402	0.0968196	NA	NA	-	NA	NA	NA	NA
								0.3859865				NA
preserved ~ pos	3262.394	2813900	0.0800	1073.920	0.0871	NA	NA	-	NA	NA	NA	NA
								0.4207398				NA
preserved ~ stimlen * log_freq	3548.322	2841900	0.0800	1073.969	0.3565141	NA	NA	NA	NA	NA	NA	NA
						0.2911978	0.0328596					
preserved ~ stimlen + log_freq	3550.327	9910300	0.0800	1073.399	0.0855372	NA	NA	NA	NA	NA	NA	NA
						0.2916508						
preserved ~ stimlen	3562.336	6910300	0.0800	1073.698	0.3158916	NA	NA	NA	NA	NA	NA	NA
preserved ~ 1	3702.476	8601900	0.0800	1073.985	NA	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * log_freq + I(pos^2) + pos"
```

```
print(BestFLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",  
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)          stimlen          log_freq          I(pos^2)          pos  stimlen:log_freq  
##      4.91590        -0.09622          0.33139          0.04095        -0.78232          -0.03286
```

```
##
```

```
## Degrees of Freedom: 4211 Total (i.e. Null); 4206 Residual
```

```
## Null Deviance: 3701
```

```
## Residual Deviance: 3214 AIC: 3226
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
```

```
median_freq <- median(PosDat$log_freq)
```

```
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
```

```
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
```

```
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

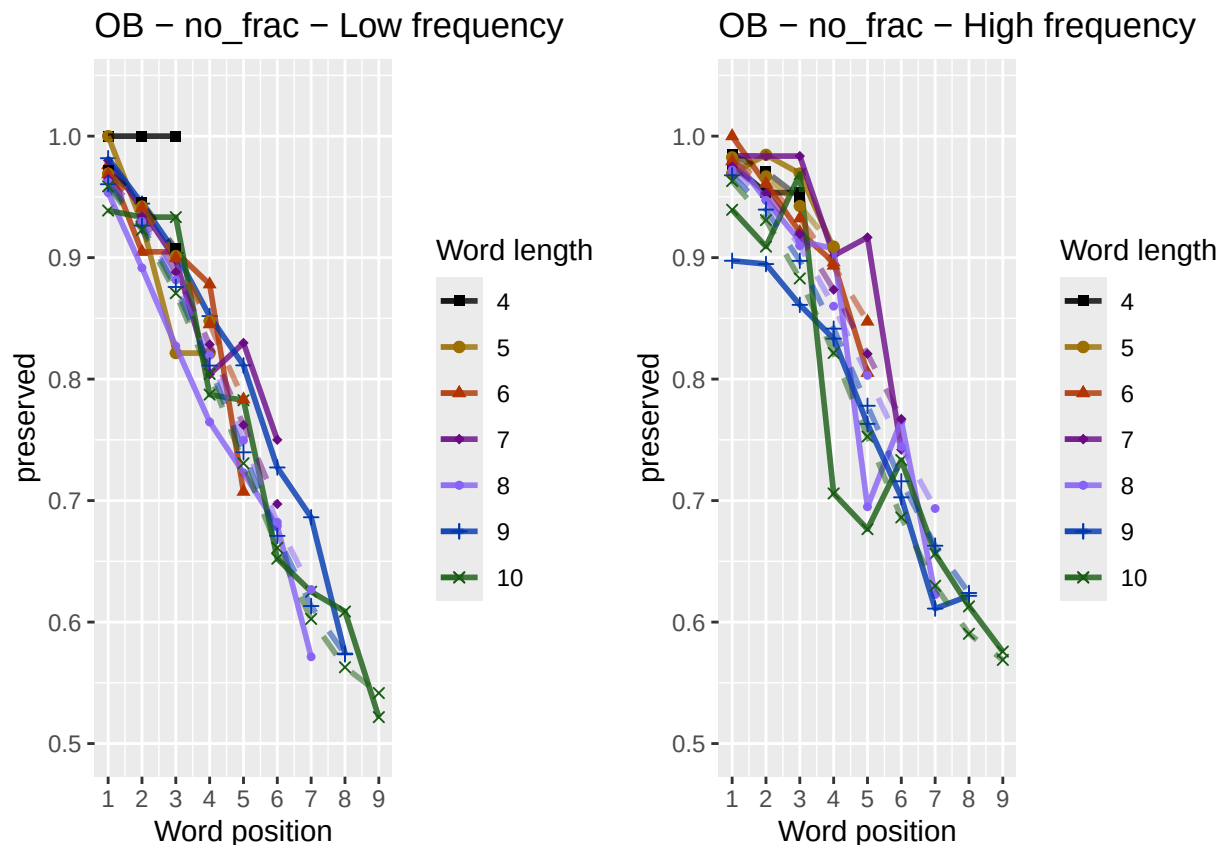
```
HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFdat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted"
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
             CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations, PosDat)
```

```

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          2.399      -1.795
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 2752  AIC: 2756
## log likelihood:  -1376.235
## Nagelkerke R2:  0.3448078
## % pres/err predicted correctly:  -784.5473
## % of predictable range [ (model-null)/(1-null) ]:  0.3053995
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##          4.25999      0.03635     -0.78085
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4209 Residual
## Null Deviance:      3701
## Residual Deviance: 3245  AIC: 3251
## log likelihood:  -1622.255
## Nagelkerke R2:  0.1755836
## % pres/err predicted correctly:  -1006.09
## % of predictable range [ (model-null)/(1-null) ]:  0.1095059
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##          3.5203     -0.4207
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3259  AIC: 3263
## log likelihood:  -1629.497
## Nagelkerke R2:  0.1702967
## % pres/err predicted correctly:  -1005.036
## % of predictable range [ (model-null)/(1-null) ]:  0.1104375
## *****
## model index:  5
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.1665      -0.3159
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3559 AIC: 3563
## log likelihood:  -1779.279
## Nagelkerke R2:  0.05676984
## % pres/err predicted correctly:  -1094.075
## % of predictable range [ (model-null)/(1-null) ]:  0.0317071
## *****
## model index:  1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.9952      -0.1284
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3665 AIC: 3669
## log likelihood:  -1832.444
## Nagelkerke R2:  0.01449158
## % pres/err predicted correctly:  -1120.859
## % of predictable range [ (model-null)/(1-null) ]:  0.008024461
## *****
## model index:  6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.66
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4211 Residual
## Null Deviance:      3701
## Residual Deviance: 3701 AIC: 3703
## log likelihood:  -1850.363
## Nagelkerke R2:  -3.797957e-16
## % pres/err predicted correctly:  -1129.934
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
BestMEModel<-MRes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MRes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MRes$Model,

```



```

      AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2756.4700	0.0000	1	1	0.3448078	2.398744	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	3250.5114	494.0407	0	0	0.1755834	4.259987	NA	NA	0.036347	-	NA
preserved ~ pos	3262.9940	66.5241	0	0	0.1702963	3.520265	NA	NA	NA	-	NA
preserved ~ stimlen	3562.5578	806.0872	0	0	0.0567698	1.166546	NA	NA	NA	NA	-
preserved ~ CumPres	3668.8891	12.4185	0	0	0.0144916	0.995189	-	NA	NA	NA	NA
preserved ~ 1	3702.7269	46.2562	0	0	0.0000000	0.659854	NA	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random average"),

```

```

                                AIC=c(mean(RndModelAIC))))
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEMModelRndDF,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
                                         RunLabel, "_", CurTask,
                                         "_syllable_component_summary.csv"), row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.8589512	553
O	0.8167939	1965
P	1.0000000	36
S	0.7603306	242
V	0.8750000	1416

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.405      -1.953
##
## Degrees of Freedom: 3933 Total (i.e. Null); 3932 Residual
## Null Deviance:      3411
## Residual Deviance: 2522 AIC: 2526
## log likelihood: -1260.838
## Nagelkerke R2: 0.3489682

```

```

## % pres/err predicted correctly: -713.8441
## % of predictable range [ (model-null)/(1-null) ]: 0.3111363
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 4.55634 0.04079 -0.85684
##
## Degrees of Freedom: 3933 Total (i.e. Null); 3931 Residual
## Null Deviance: 3411
## Residual Deviance: 2946 AIC: 2952
## log likelihood: -1472.794
## Nagelkerke R2: 0.1924454
## % pres/err predicted correctly: -912.7152
## % of predictable range [ (model-null)/(1-null) ]: 0.1194929
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.6933 -0.4472
##
## Degrees of Freedom: 3933 Total (i.e. Null); 3932 Residual
## Null Deviance: 3411
## Residual Deviance: 2961 AIC: 2965
## log likelihood: -1480.723
## Nagelkerke R2: 0.1862565
## % pres/err predicted correctly: -911.7508
## % of predictable range [ (model-null)/(1-null) ]: 0.1204223
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 4.2099 -0.3177
##
## Degrees of Freedom: 3933 Total (i.e. Null); 3932 Residual
## Null Deviance: 3411
## Residual Deviance: 3279 AIC: 3283
## log likelihood: -1639.717
## Nagelkerke R2: 0.05674092
## % pres/err predicted correctly: -1004.108
## % of predictable range [ (model-null)/(1-null) ]: 0.03142145
## *****

```

```
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0365      -0.1419
##
## Degrees of Freedom: 3933 Total (i.e. Null); 3932 Residual
## Null Deviance:      3411
## Residual Deviance: 3374 AIC: 3378
## log likelihood: -1686.844
## Nagelkerke R2: 0.01629596
## % pres/err predicted correctly: -1027.327
## % of predictable range [ (model-null)/(1-null) ]: 0.009046371
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.686
##
## Degrees of Freedom: 3933 Total (i.e. Null); 3933 Residual
## Null Deviance:      3411
## Residual Deviance: 3411 AIC: 3413
## log likelihood: -1705.518
## Nagelkerke R2: -3.829562e-16
## % pres/err predicted correctly: -1036.715
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir, CurPat, "_", RunLabel, "_",
                  CurTask,
                  "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2525.677	0.0000	1	1	0.3489682	405096	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2951.588	425.9114	0	0	0.1924454	54.556340	NA	NA	0.0407914	-	NA
preserved ~ pos	2965.446	439.7694	0	0	0.1862563	63.693261	NA	NA	NA	-	NA
preserved ~ stimlen	3283.433	57.7566	0	0	0.0567409	4.209945	NA	NA	NA	NA	-
preserved ~ CumPres	3377.688	52.0118	0	0	0.0162960	0.036532	-	NA	NA	NA	NA
preserved ~ 1	3413.036	87.3596	0	0	0.0000000	0.685796	NA	NA	NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
-------	-----	----------	--------	-------	-------	-------------	---------	--------	----------	-----	---------

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.320      -2.135
##
## Degrees of Freedom: 3380 Total (i.e. Null); 3379 Residual
## Null Deviance:      2960
## Residual Deviance: 2253 AIC: 2257
## log likelihood: -1126.336
## Nagelkerke R2: 0.323566
## % pres/err predicted correctly: -643.0675
## % of predictable range [ (model-null)/(1-null) ]: 0.2870769
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      4.56171      0.04361     -0.87858
##
## Degrees of Freedom: 3380 Total (i.e. Null); 3378 Residual
## Null Deviance:      2960
## Residual Deviance: 2543 AIC: 2549
## log likelihood: -1271.433
## Nagelkerke R2: 0.1989245
## % pres/err predicted correctly: -791.7865
## % of predictable range [ (model-null)/(1-null) ]: 0.1224587
## *****
## model index: 4
##
```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.6501      -0.4416
##
## Degrees of Freedom: 3380 Total (i.e. Null); 3379 Residual
## Null Deviance:      2960
## Residual Deviance: 2560 AIC: 2564
## log likelihood: -1279.982
## Nagelkerke R2: 0.1912416
## % pres/err predicted correctly: -791.1331
## % of predictable range [ (model-null)/(1-null) ]: 0.123182
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.0765      -0.3044
##
## Degrees of Freedom: 3380 Total (i.e. Null); 3379 Residual
## Null Deviance:      2960
## Residual Deviance: 2852 AIC: 2856
## log likelihood: -1426.142
## Nagelkerke R2: 0.05369718
## % pres/err predicted correctly: -875.5196
## % of predictable range [ (model-null)/(1-null) ]: 0.02977399
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.1746      -0.2375
##
## Degrees of Freedom: 3380 Total (i.e. Null); 3379 Residual
## Null Deviance:      2960
## Residual Deviance: 2891 AIC: 2895
## log likelihood: -1445.486
## Nagelkerke R2: 0.03458553
## % pres/err predicted correctly: -884.362
## % of predictable range [ (model-null)/(1-null) ]: 0.01998625
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##

```

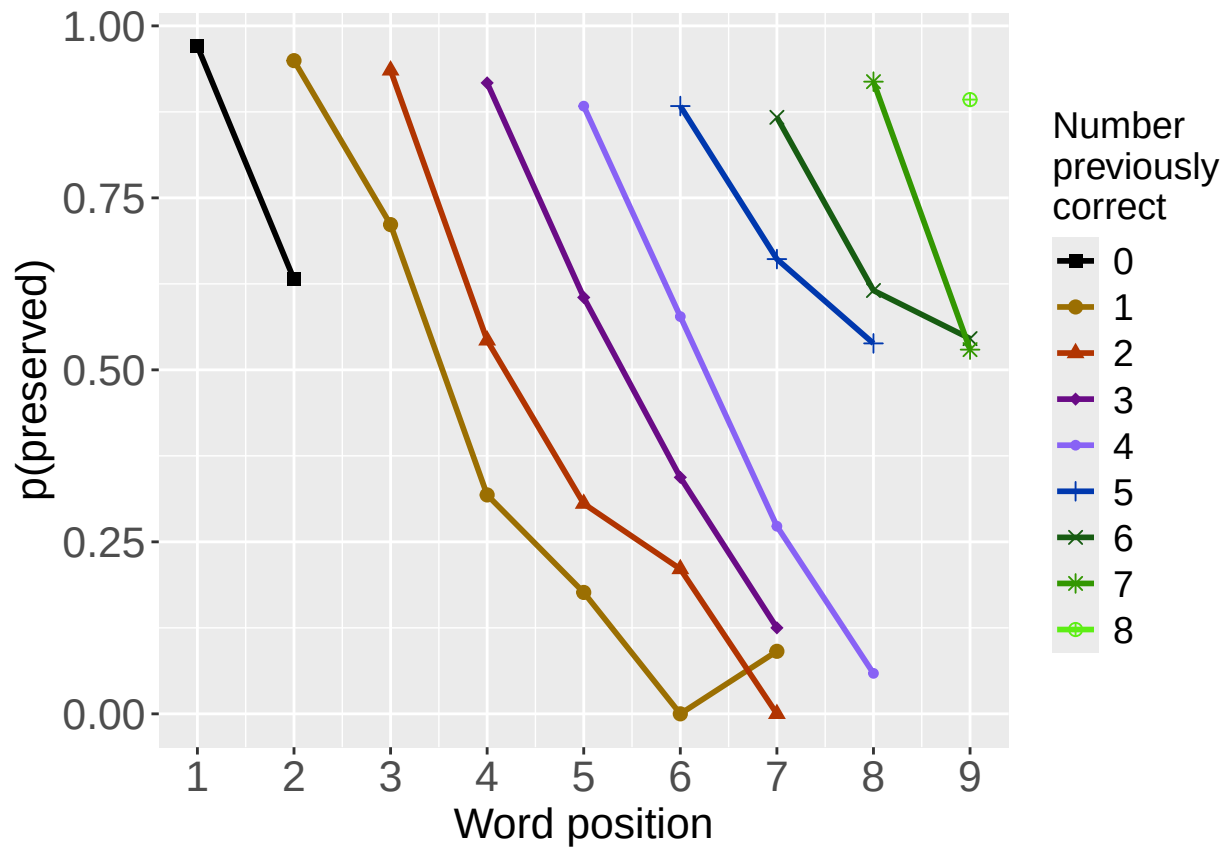
```
## Coefficients:
## (Intercept)
##      1.667
##
## Degrees of Freedom: 3380 Total (i.e. Null);  3380 Residual
## Null Deviance:      2960
## Residual Deviance: 2960  AIC: 2962
## log likelihood:  -1479.94
## Nagelkerke R2:  -3.806543e-16
## % pres/err predicted correctly:  -902.4179
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErrI(pos^2)	pos	stimlen
preserved ~ CumErr	2256.672	0.0000	1	1	0.3235666	0.320105	NA	- 2.13531	NA	NA
preserved ~ (I(pos^2) + pos)	2548.866	292.1939	0	0	0.1989244	0.561707	NA	NA	0.0436113	NA
preserved ~ pos	2563.964	307.2920	0	0	0.1912416	0.650135	NA	NA	NA	NA
preserved ~ stimlen	2856.283	599.6113	0	0	0.0536974	0.076517	NA	NA	NA	- 0.3043697
preserved ~ CumPres	2894.975	638.3003	0	0	0.0345852	0.174648	- 0.2374574	NA	NA	NA
preserved ~ 1	2961.880	705.2079	0	0	0.0000000	0.666969	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

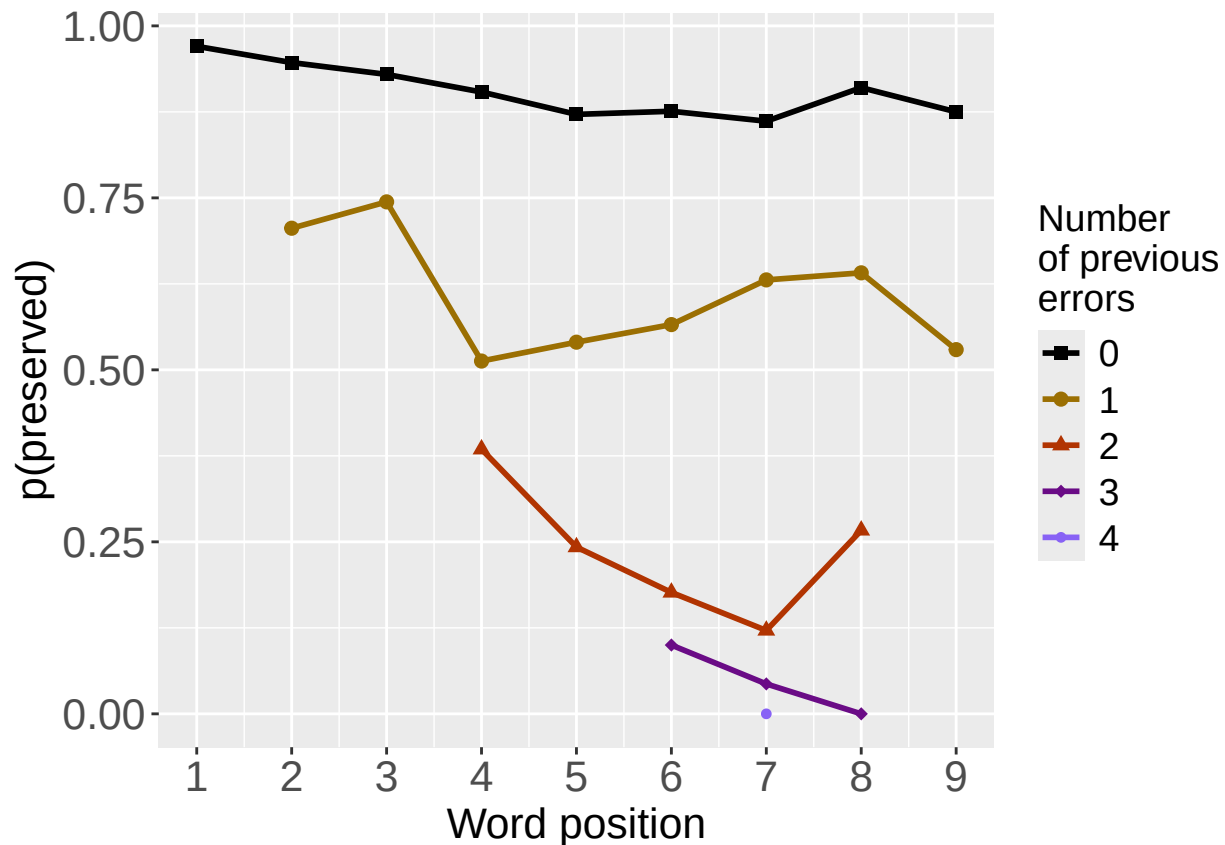
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

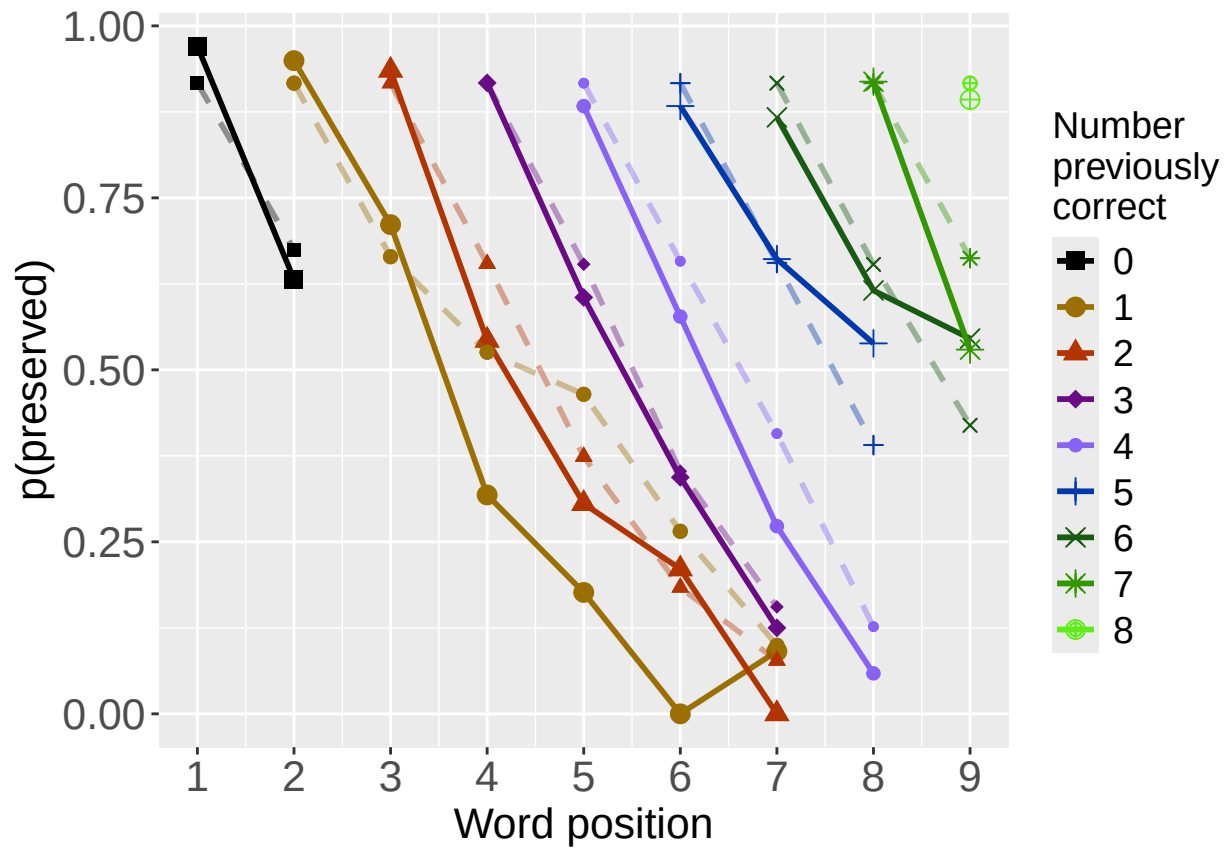
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

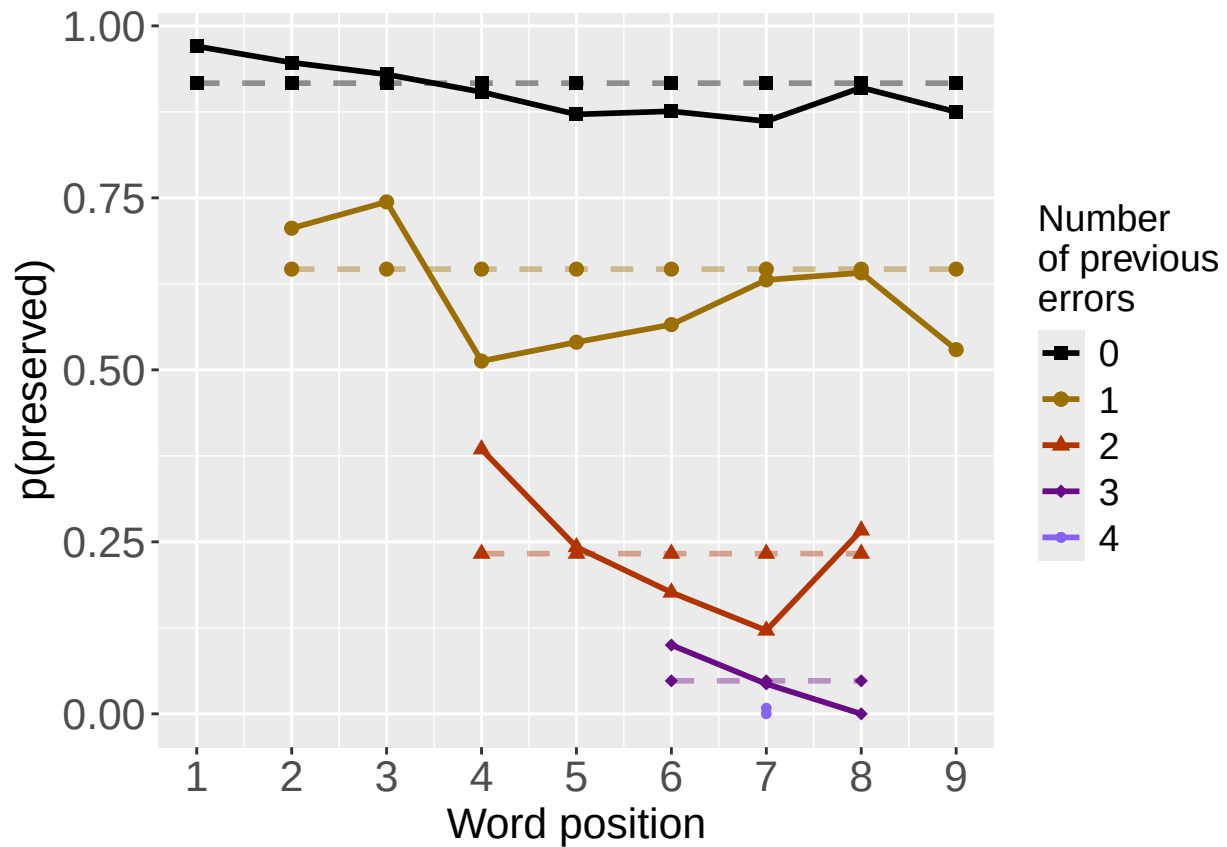
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##    4.18124    -1.53628     0.06075    -0.75560
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4208 Residual
## Null Deviance:      3701
## Residual Deviance: 2672  AIC: 2680
## log likelihood:  -1335.755
## Nagelkerke R2:  0.3708064
## % pres/err predicted correctly:  -772.0458
## % of predictable range [ (model-null)/(1-null) ]:  0.3164536
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.399      -1.795
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 2752  AIC: 2756
## log likelihood:  -1376.235
## Nagelkerke R2:  0.3448078
## % pres/err predicted correctly:  -784.5473
## % of predictable range [ (model-null)/(1-null) ]:  0.3053995
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      4.25999      0.03635      -0.78085
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4209 Residual
## Null Deviance:      3701
## Residual Deviance: 3245  AIC: 3251
## log likelihood:  -1622.255
## Nagelkerke R2:  0.1755836
## % pres/err predicted correctly:  -1006.09
## % of predictable range [ (model-null)/(1-null) ]:  0.1095059
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	2679.511	0.00000	1	1	0.3708064	4.181241	-1.536284	0.060749	-0.7555968
preserved ~ CumErr	2756.470	76.95954	0	0	0.3448078	2.398744	-1.795432	NA	NA
preserved ~ I(pos^2) + pos	3250.511	571.00020	0	0	0.1755836	4.259987	NA	0.036347	-0.7808527

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 3.4065 -1.7224 -0.1316
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4209 Residual
## Null Deviance: 3701
## Residual Deviance: 2735 AIC: 2741
## log likelihood: -1367.62
## Nagelkerke R2: 0.3503829
## % pres/err predicted correctly: -783.4512
## % of predictable range [ (model-null)/(1-null) ]: 0.3063687
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.399 -1.795
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4210 Residual
## Null Deviance: 3701
## Residual Deviance: 2752 AIC: 2756
## log likelihood: -1376.235
## Nagelkerke R2: 0.3448078
## % pres/err predicted correctly: -784.5473
## % of predictable range [ (model-null)/(1-null) ]: 0.3053995
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      4.1665      -0.3159
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3559  AIC: 3563
## log likelihood:  -1779.279
## Nagelkerke R2:  0.05676984
## % pres/err predicted correctly:  -1094.075
## % of predictable range [ (model-null)/(1-null) ]:  0.0317071
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	2741.240	0.0000	1.0000000	0.9995073	0.3503829	3.406475	- 1.722372	- 0.1315538
preserved ~ CumErr	2756.470	15.2303	0.0004929	0.0004927	0.3448078	2.398744	- 1.795432	NA
preserved ~ stimlen	3562.557	821.3175	0.0000000	0.0000000	0.0567698	4.166546	NA	- 0.3158916

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.7521      -1.7640      -0.1331
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4209 Residual
## Null Deviance:      3701
## Residual Deviance: 2726  AIC: 2732
## log likelihood:  -1362.954
## Nagelkerke R2:  0.353393
## % pres/err predicted correctly:  -781.7526
## % of predictable range [ (model-null)/(1-null) ]:  0.3078707
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.399      -1.795
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 2752  AIC: 2756
## log likelihood:  -1376.235
## Nagelkerke R2:  0.3448078
## % pres/err predicted correctly:  -784.5473
## % of predictable range [ (model-null)/(1-null) ]:  0.3053995
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.9952      -0.1284
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3665  AIC: 3669
## log likelihood:  -1832.444
## Nagelkerke R2:  0.01449158
## % pres/err predicted correctly:  -1120.859
## % of predictable range [ (model-null)/(1-null) ]:  0.008024461
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	2731.908	0.00000	1.0e+00	0.9999954	0.3533930	2.752121	- 1.763958	- 0.1330963
preserved ~ CumErr	2756.470	24.56248	4.6e-06	0.0000046	0.3448078	2.398744	- 1.795432	NA
preserved ~ CumPres	3668.889	936.98095	0.0e+00	0.0000000	0.0144916	1.995189	NA	- 0.1283603

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      3.0954      -1.5445      -0.1851
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4209 Residual
## Null Deviance:      3701
## Residual Deviance: 2701 AIC: 2707
## log likelihood: -1350.746
## Nagelkerke R2: 0.3612364
## % pres/err predicted correctly: -776.664
## % of predictable range [ (model-null)/(1-null) ]: 0.3123701
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.399      -1.795
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4210 Residual
## Null Deviance:      3701
## Residual Deviance: 2752 AIC: 2756
## log likelihood: -1376.235
## Nagelkerke R2: 0.3448078
## % pres/err predicted correctly: -784.5473
## % of predictable range [ (model-null)/(1-null) ]: 0.3053995
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.5203      -0.4207
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4210 Residual
## Null Deviance:      3701
## Residual Deviance: 3259 AIC: 3263
## log likelihood: -1629.497
## Nagelkerke R2: 0.1702967
## % pres/err predicted correctly: -1005.036
## % of predictable range [ (model-null)/(1-null) ]: 0.1104375
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	2707.493	0.0000	1	1	0.3612364	3.095352	-	-
+ pos							1.544504	0.1850951
preserved ~ CumErr	2756.470	48.9775	0	0	0.3448078	2.398744	-	NA
							1.795432	
preserved ~ pos	3262.994	555.5016	0	0	0.1702967	3.520265	NA	-
								0.4207398

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + I(pos^2) + pos	2679.510	0.00000	1.000000	0.000000	0.3708064	4.181241	-	0.060749	-	NA	NA
							1.536284		0.7555968		
preserved ~ CumErr + pos	2707.498	0.00000	1.000000	0.000000	0.3612363	3.095352	-	NA	-	NA	NA
							1.544504		0.1850951		
preserved ~ CumErr + CumPres	2731.908	0.00000	1.000000	0.999999	0.3533393	3.0752121	-	NA	NA	NA	-
							1.763958				0.1330963
preserved ~ CumErr + stimlen	2741.240	0.00000	1.000000	0.9995073	0.3503829	3.0406475	-	NA	NA	-	NA
							1.722372		0.1315538		
preserved ~ CumErr	2756.470	6.95954	0.000000	0.000000	0.3448078	2.398744	-	NA	NA	NA	NA
							1.795432				
preserved ~ CumErr	2756.470	5.23030	0.000492	0.000492	0.3448078	2.398744	-	NA	NA	NA	NA
							1.795432				
preserved ~ CumErr	2756.470	4.56248	0.000004	0.000004	0.3448078	2.398744	-	NA	NA	NA	NA
							1.795432				
preserved ~ CumErr	2756.470	8.97750	0.000000	0.000000	0.3448078	2.398744	-	NA	NA	NA	NA
							1.795432				
preserved ~ I(pos^2) + pos	3250.515	71.00020	0.000000	0.000000	0.1755836	3.259987	NA	0.036347	-	NA	NA
								0.7808527			
preserved ~ pos	3262.994	55.50160	0.000000	0.000000	0.1702967	3.520265	NA	NA	-	NA	NA
								0.4207398			
preserved ~ stimlen	3562.558	21.31745	0.000000	0.000000	0.0567698	3.166546	NA	NA	NA	-	NA
									0.3158916		
preserved ~ CumPres	3668.889	36.98095	0.000000	0.000000	0.0144916	3.995189	NA	NA	NA	NA	-
											0.1283603

explore influence of frequency and length

```
if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)          pos      stimlen      log_freq
##      4.66698      -1.51614      0.06459      -0.76373      -0.06780      0.07207
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4206 Residual
## Null Deviance:      3701
## Residual Deviance: 2659 AIC: 2671
## log likelihood: -1329.309
## Nagelkerke R2: 0.3749007
## % pres/err predicted correctly: -770.5215
## % of predictable range [ (model-null)/(1-null) ]: 0.3178014
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)          pos      log_freq
##      4.17014      -1.52366      0.06137      -0.75424      0.08380
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4207 Residual
## Null Deviance:      3701
## Residual Deviance: 2662 AIC: 2672
## log likelihood: -1330.971
## Nagelkerke R2: 0.3738465
## % pres/err predicted correctly: -770.1715
## % of predictable range [ (model-null)/(1-null) ]: 0.318111
## *****
## model index: 4

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos      stimlen
##      4.83468      -1.52460      0.06513      -0.76746      -0.08976
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4207 Residual
## Null Deviance:      3701
## Residual Deviance: 2665 AIC: 2675
## log likelihood: -1332.658
## Nagelkerke R2: 0.3727753
## % pres/err predicted correctly: -772.1358
## % of predictable range [ (model-null)/(1-null) ]: 0.3163741
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos
##      4.18124      -1.53628      0.06075      -0.75560
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4208 Residual
## Null Deviance:      3701
## Residual Deviance: 2672 AIC: 2680
## log likelihood: -1335.755
## Nagelkerke R2: 0.3708064
## % pres/err predicted correctly: -772.0458
## % of predictable range [ (model-null)/(1-null) ]: 0.3164536
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.66
##
## Degrees of Freedom: 4211 Total (i.e. Null);  4211 Residual
## Null Deviance:      3701
## Residual Deviance: 3701 AIC: 3703
## log likelihood: -1850.363
## Nagelkerke R2: -3.797957e-16
## % pres/err predicted correctly: -1129.934
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	log_freq	stimlen
preserved ~ CumErr + I(pos^2) + pos + stimlen + log_freq	2670.618	0.000000	1.000000	0.160994	0.374900	0.766978	-	0.0645858	-	0.0720746	-
							1.516145		0.7637297		0.0678022
preserved ~ CumErr + I(pos^2) + pos + log_freq	2671.941	1.323620	0.515916	0.317856	0.373846	0.5170145	-	0.0613677	-	0.0837955	NA
							1.523660		0.7542422		
preserved ~ CumErr + I(pos^2) + pos + stimlen	2675.315	6.697716	0.095478	0.205882	0.137277	0.334684	-	0.0651253	-	NA	-
							1.524604		0.7674626		0.0897598
preserved ~ CumErr + I(pos^2) + pos	2679.511	8.892927	0.011719	0.007220	0.037080	0.4181241	-	0.0607490	-	NA	NA
							1.536284		0.7555968		
preserved ~ 1	3702.726	1032.108	0.000000	0.000000	0.000000	0.000000	NA	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + I(pos^2) + pos + stimlen + log_freq
##      Df Deviance   AIC
## CumErr  1   3217.3 3227.3
## pos     1   2710.2 2720.2
## I(pos^2) 1   2691.7 2701.7
## log_freq 1   2665.3 2675.3
## stimlen  1   2661.9 2671.9
## <none>    1   2658.6 2670.6

```

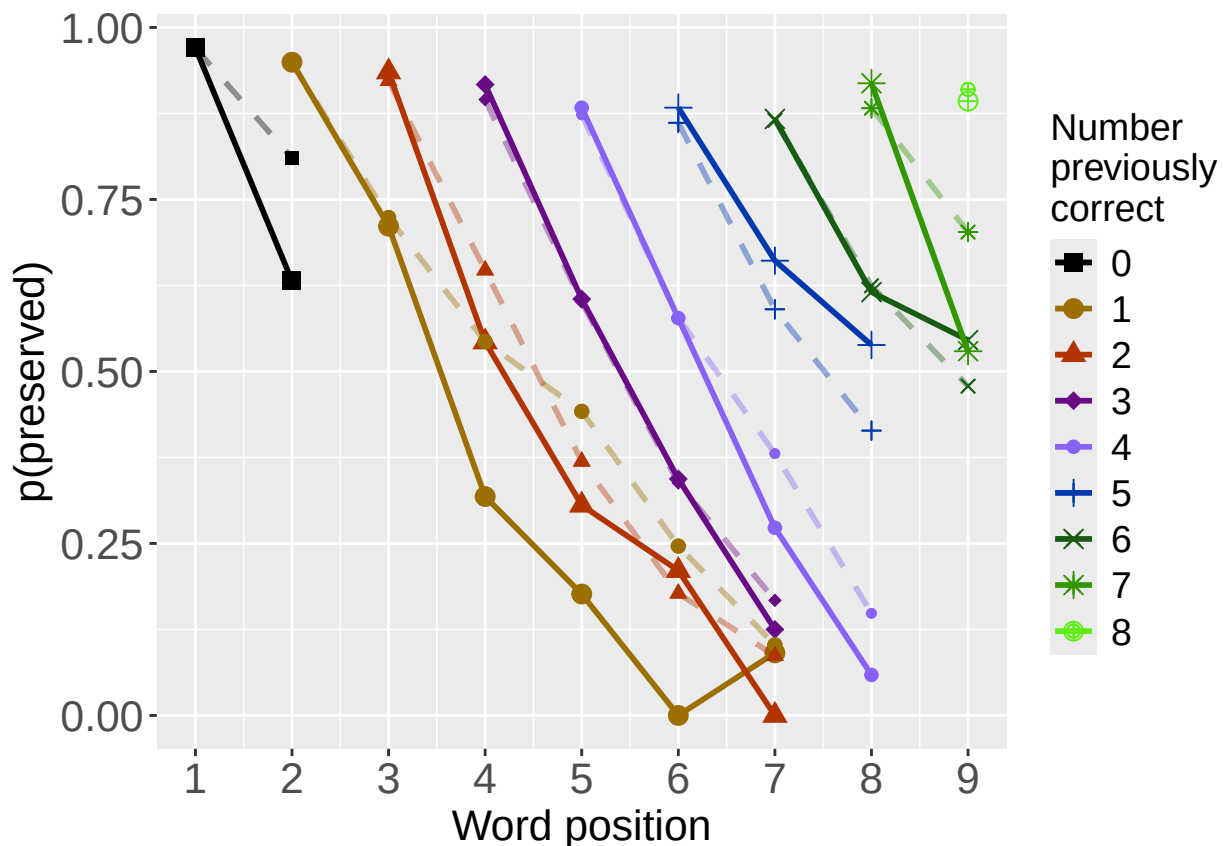
```
#####
# Single deletions from best model
#####

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

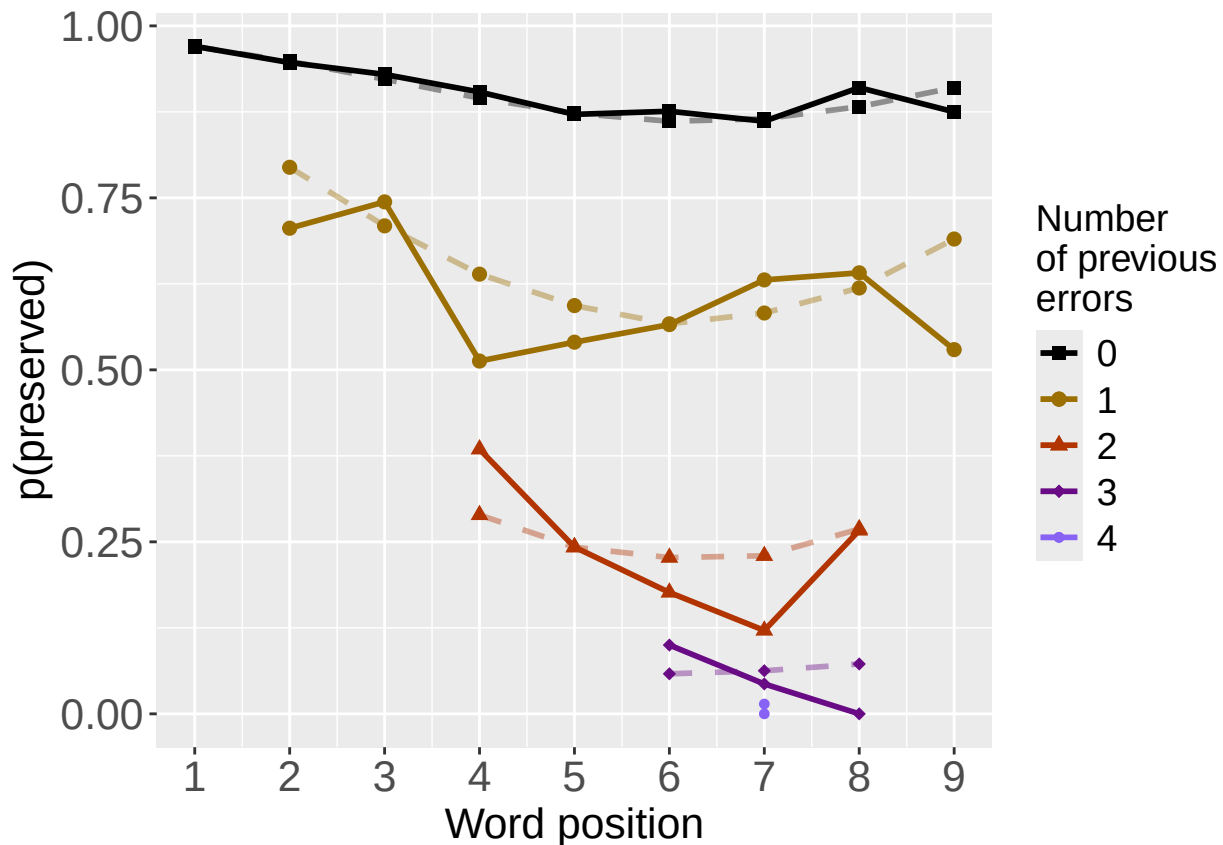
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.399      -1.795
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4210 Residual
## Null Deviance: 3701
## Residual Deviance: 2752 AIC: 2756
## log likelihood: -1376.235
## Nagelkerke R2: 0.3448078
## % pres/err predicted correctly: -784.5473
## % of predictable range [ (model-null)/(1-null) ]: 0.3053995

```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      3.0954      -1.5445      -0.1851
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4209 Residual
## Null Deviance:      3701
## Residual Deviance: 2701 AIC: 2707
## log likelihood: -1350.746
## Nagelkerke R2: 0.3612364
## % pres/err predicted correctly: -776.664
## % of predictable range [ (model-null)/(1-null) ]: 0.3123701
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos      I(pos^2)
##      4.18124      -1.53628      -0.75560      0.06075
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4208 Residual
## Null Deviance:      3701
## Residual Deviance: 2672 AIC: 2680
## log likelihood: -1335.755
## Nagelkerke R2: 0.3708064
## % pres/err predicted correctly: -772.0458
## % of predictable range [ (model-null)/(1-null) ]: 0.3164536
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos      I(pos^2)      log_freq
##      4.17014      -1.52366      -0.75424      0.06137      0.08380
##
## Degrees of Freedom: 4211 Total (i.e. Null); 4207 Residual
## Null Deviance:      3701
## Residual Deviance: 2662 AIC: 2672
## log likelihood: -1330.971
## Nagelkerke R2: 0.3738465
## % pres/err predicted correctly: -770.1715
## % of predictable range [ (model-null)/(1-null) ]: 0.318111
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

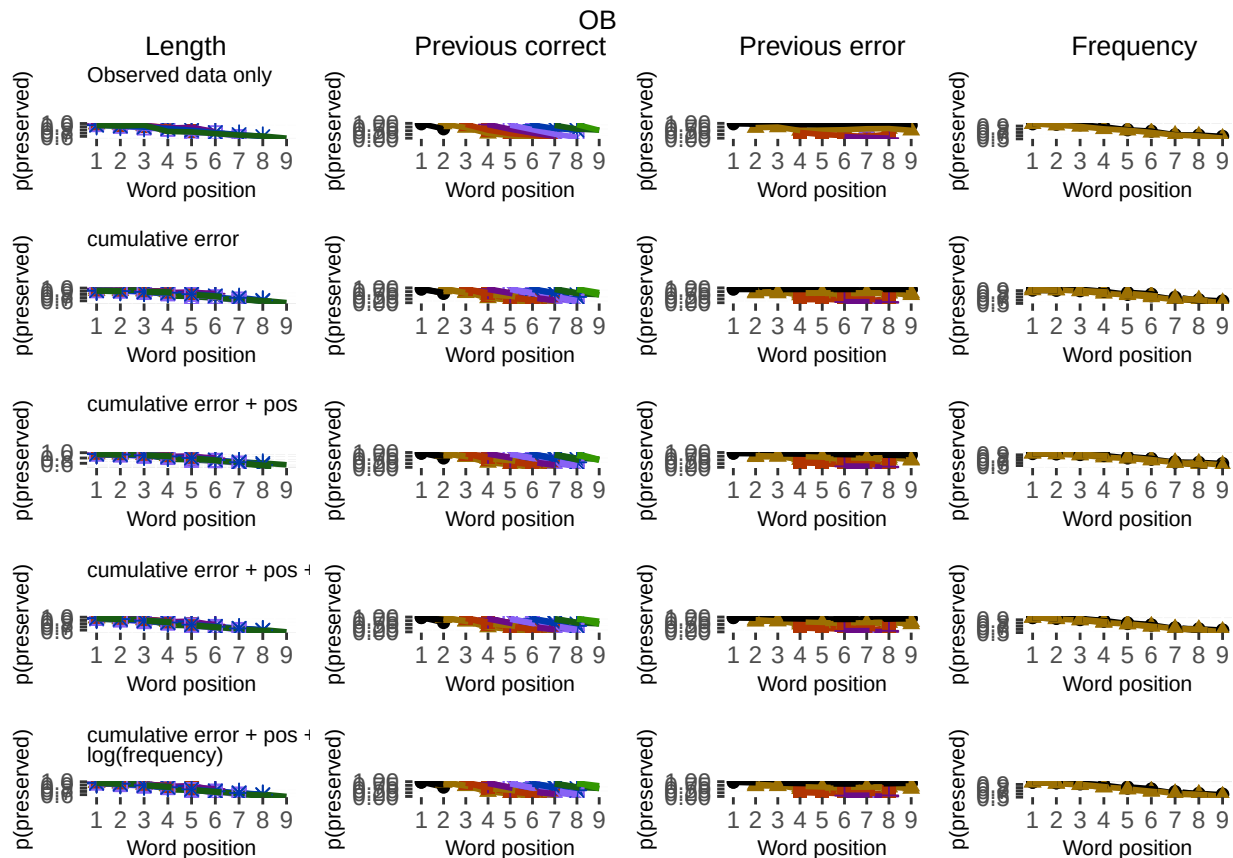
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	I(pos^2)	pos	stimlen	log_freq
McFadden	0.1837807	0.0363508	0.0461460	0.0104646	0.0048536

	CumErr	I(pos^2)	pos	stimlen	log_freq
SquaredCorrelation	0.1393312	0.0297576	0.0372805	0.0088708	0.0039427
Nagelkerke	0.1393312	0.0297576	0.0372805	0.0088708	0.0039427
Estrella	0.1652328	0.0322905	0.0410788	0.0092465	0.0043202

	deviance	deviance_explained
CumErr + pos + I(pos ²) + log_freq	2661.941	1038.7850
CumErr + pos + I(pos ²)	2671.511	1029.2157
CumErr + pos	2701.493	999.2337
CumErr	2752.470	948.2562
null	3700.726	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained
## CumErr + pos + I(pos^2) + log_freq CumErr + pos + I(pos^2) + log_freq 2661.941      1038.7850      28.06976
## CumErr + pos + I(pos^2)              CumErr + pos + I(pos^2) 2671.511      1029.2157      27.81118
## CumErr + pos                        CumErr + pos 2701.493      999.2337      27.00101
## CumErr                            CumErr 2752.470      948.2562      25.62352
## null                                null 3700.726      0.0000      0.00000
##                                percent_of_explained_deviance increment_in_explained
## CumErr + pos + I(pos^2) + log_freq      100.00000      0.9212018
## CumErr + pos + I(pos^2)                  99.07880      2.8862602
## CumErr + pos                            96.19254      4.9074158
## CumErr                                  91.28512      91.2851221
## null                                    NA      0.0000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + pos + I(pos ²) + log_freq	28.06976	100.00000	0.9212018
CumErr + pos + I(pos ²)	27.81118	99.07880	2.8862602
CumErr + pos	27.00101	96.19254	4.9074158
CumErr	25.62352	91.28512	91.2851221
null	0.00000	NA	0.0000000

```

                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.63568493
## I(pos^2)  0.13576601
## pos       0.17008850
## stimlen   0.04047218
## log_freq  0.01798838

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.

```

```

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)

sse_table

##
## 1          model p_accounted_for model_deviance diff_CumErr diff_CumErr+pos diff_CumErr+pos+I(pos^2)
## 2          preserved ~ CumErr      0.9758478      2752.470 0.000000000      -0.007004096      -0.012417441
## 2          preserved ~ CumErr+pos    0.9828519      2701.493 0.007004096       0.000000000      -0.005413345

```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.9758478	2752.470
preserved ~ CumErr+pos	0.9828519	2701.493
preserved ~ CumErr+pos+I(pos ²)	0.9882652	2671.511
preserved ~ CumErr+pos+I(pos ²)+log_freq	0.9884973	2661.941

model	diff_CumErr	diff_CumErr+pos	diff_CumErr+pos+I(pos ²)	diff_CumErr+pos+I(pos ²)+log_freq
preserved ~ CumErr	0.0000000	-0.0070041	-0.0124174	-0.0126495
preserved ~ CumErr+pos	0.0070041	0.0000000	-0.0054133	-0.0056454
preserved ~ CumErr+pos+I(pos ²)	0.0124174	0.0054133	0.0000000	-0.0002321
preserved ~ CumErr+pos+I(pos ²)+log_freq	0.0126495	0.0056454	0.0002321	0.0000000

```
## 3      preserved ~ CumErr+pos+I(pos^2)      0.9882652      2671.511 0.012417441      0.005413345      0.000000000
## 4 preserved ~ CumErr+pos+I(pos^2)+log_freq      0.9884973      2661.941 0.012649493      0.005645397      0.000232052
## diff_CumErr+pos+I(pos^2)+log_freq
## 1      -0.012649493
## 2      -0.005645397
## 3      -0.000232052
## 4      0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```